
Batch 4 — PHP Backend Foundation

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:05

Batch 4 — PHP Backend Foundation

Meat Lovers CIMS powered by YohPal

4A.

backend/public/index.php

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/../bootstrap/app.php';

$router = require __DIR__ . '/../routes/api.php';

$router->dispatch(
    $_SERVER['REQUEST_METHOD'],
    parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH)
);
```

4B.

backend/bootstrap/app.php

```
<?php

declare(strict_types=1);

spl_autoload_register(function ($class) {
    $baseDir = __DIR__ . '/../app/';
    $class = str_replace('\\', '/', $class);
    $file = $baseDir . $class . '.php';

    if (file_exists($file)) {
        require_once $file;
    }
});

require_once __DIR__ . '/../config/database.php';

header('Content-Type: application/json');
```

4C.

backend/config/database.php

```
<?php

declare(strict_types=1);

use Support\Database;

Database::configure([
    'host' => $_ENV['DB_HOST'] ?? '127.0.0.1',
    'port' => $_ENV['DB_PORT'] ?? '3306',
    'database' => $_ENV['DB_DATABASE'] ?? 'meat_lovers_cims',
    'username' => $_ENV['DB_USERNAME'] ?? 'root',
    'password' => $_ENV['DB_PASSWORD'] ?? '',
]);
```

4D.

backend/app/Support/Database.php

```
<?php

declare(strict_types=1);

namespace Support;

use PDO;

class Database
{
    private static array $config;
    private static ?PDO $connection = null;

    public static function configure(array $config): void
    {
        self::$config = $config;
    }

    public static function connection(): PDO
    {
        if (self::$connection === null) {
            $dsn = sprintf(
                'mysql:host=%s;port=%s;dbname=%s;charset=utf8mb4',
                self::$config['host'],
                self::$config['port'],
                self::$config['database']
            );

            self::$connection = new PDO(
                $dsn,
                self::$config['username'],
                self::$config['password'],
                [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                ]
            );
        }

        return self::$connection;
    }
}
```

4E.

backend/app/Support/Response.php

```
<?php

declare(strict_types=1);

namespace Support;

class Response
{
    public static function json(array $data, int $status = 200): void
    {
        http_response_code($status);
        echo json_encode($data);
        exit;
    }

    public static function success(array $data = [], string $message = 'Success'): void
    {
        self::json([
            'success' => true,
            'message' => $message,
            'data' => $data,
        ]);
    }

    public static function error(string $message, int $status = 400): void
    {
        self::json([
            'success' => false,
            'message' => $message,
        ], $status);
    }
}
```

4F.

backend/app/Support/Request.php

```
<?php

declare(strict_types=1);

namespace Support;

class Request
{
    public static function body(): array
    {
        $raw = file_get_contents('php://input');
        $data = json_decode($raw, true);

        return is_array($data) ? $data : [];
    }

    public static function input(string $key, mixed $default = null): mixed
    {
        $body = self::body();
        return $body[$key] ?? $_POST[$key] ?? $_GET[$key] ?? $default;
    }
}
```

4G.

backend/app/Support/Router.php

```
<?php

declare(strict_types=1);

namespace Support;

class Router
{
    private array $routes = [];

    public function get(string $path, array $handler): void
    {
        $this->routes['GET'][$path] = $handler;
    }

    public function post(string $path, array $handler): void
    {
        $this->routes['POST'][$path] = $handler;
    }

    public function dispatch(string $method, string $uri): void
    {
        $path = str_replace('/backend/public', '', $uri);

        foreach ($this->routes[$method] ?? [] as $route => $handler) {
            $pattern = preg_replace('/\{[a-zA-Z_]+\}/', '([0-9]+)', $route);
            $pattern = '#^'. $pattern . '$#';

            if (preg_match($pattern, $path, $matches)) {
                array_shift($matches);

                [$class, $methodName] = $handler;
                $controller = new $class();

                call_user_func_array([$controller, $methodName], $matches);
                return;
            }
        }

        Response::error('Route not found', 404);
    }
}
```

4H.

backend/routes/api.php

```
<?php

declare(strict_types=1);
```

```

use Support\Router;
use Controllers\AuthController;
use Controllers\ProductsController;
use Controllers\SuppliersController;
use Controllers\CustomersController;
use Controllers\InventoryController;
use Controllers\OrdersController;
use Controllers\PaymentsController;
use Controllers\ReportsController;
use Controllers\MonitoringController;

$router = new Router();

$router->post('/api/auth/login', [AuthController::class, 'login']);

$router->get('/api/products', [ProductsController::class, 'index']);
$router->post('/api/products', [ProductsController::class, 'store']);

$router->get('/api/suppliers', [SuppliersController::class, 'index']);
$router->post('/api/suppliers', [SuppliersController::class, 'store']);

$router->get('/api/customers', [CustomersController::class, 'index']);
$router->post('/api/customers', [CustomersController::class, 'store']);

$router->get('/api/inventory/stock-summary', [InventoryController::class, 'stockSummary']);
$router->post('/api/inventory/stock-in', [InventoryController::class, 'stockIn']);

$router->get('/api/orders', [OrdersController::class, 'index']);
$router->post('/api/orders', [OrdersController::class, 'store']);

$router->post('/api/payments/cash-settle/{id}', [PaymentsController::class, 'cashSettle']);

$router->get('/api/reports/daily', [ReportsController::class, 'daily']);

$router->get('/api/monitoring/overview', [MonitoringController::class, 'overview']);

return $router;

```

41.

backend/app/Controllers/AuthController.php

```

<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class AuthController
{
    public function login(): void
    {
        $email = Request::input('email');
        $password = Request::input('password');

        $stmt = Database::connection()->prepare(
            'SELECT * FROM users WHERE email = :email AND is_active = 1 LIMIT 1'
        );

        $stmt->execute(['email' => $email]);
        $user = $stmt->fetch();

        if (!$user || !password_verify($password, $user['password_hash'])) {
            Response::error('Invalid login credentials', 401);
        }

        Response::success([
            'token' => base64_encode($user['id'] . '|' . time()),
            'user' => [
                'id' => $user['id'],
                'full_name' => $user['full_name'],
                'email' => $user['email'],
                'role' => $user['role'],
            ],
        ], 'Login successful');
    }
}

```

4J.

backend/app/Controllers/ProductsController.php

```
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class ProductsController
{
    public function index(): void
    {
        $stmt = Database::connection()->query(
            'SELECT * FROM products ORDER BY created_at DESC'
        );

        Response::success(['products' => $stmt->fetchAll()]);
    }

    public function store(): void
    {
        $data = Request::body();

        $stmt = Database::connection()->prepare(
            'INSERT INTO products
            (product_name, product_category, selling_price, cost_price, barcode, is_active)
            VALUES (:name, :category, :selling, :cost, :barcode, 1)'
        );

        $stmt->execute([
            'name' => $data['product_name'],
            'category' => $data['product_category'],
            'selling' => $data['selling_price'],
            'cost' => $data['cost_price'],
            'barcode' => $data['barcode'] ?? null,
        ]);

        Response::success([], 'Product created');
    }
}
```

4K.

backend/app/Controllers/SuppliersController.php

```
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class SuppliersController
{
    public function index(): void
    {
        $stmt = Database::connection()->query(
            'SELECT * FROM suppliers ORDER BY created_at DESC'
        );

        Response::success(['suppliers' => $stmt->fetchAll()]);
    }

    public function store(): void
    {
        $data = Request::body();

        $stmt = Database::connection()->prepare(
            'INSERT INTO suppliers
            (supplier_name, contact_person, phone, email, physical_address, supplier_type, status)
            VALUES (:name, :contact, :phone, :email, :address, :type, :status)'
        );

        $stmt->execute([
```

```

        'name' => $data['supplier_name'],
        'contact' => $data['contact_person'] ?? null,
        'phone' => $data['phone'] ?? null,
        'email' => $data['email'] ?? null,
        'address' => $data['physical_address'] ?? null,
        'type' => $data['supplier_type'],
        'status' => $data['status'] ?? 'ACTIVE',
    ]);

    Response::success([], 'Supplier created');
}
}

```

4L.

backend/app/Controllers/CustomersController.php

```

<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class CustomersController
{
    public function index(): void
    {
        $stmt = Database::connection()->query(
            'SELECT * FROM customers ORDER BY created_at DESC'
        );

        Response::success(['customers' => $stmt->fetchAll()]);
    }

    public function store(): void
    {
        $data = Request::body();

        $stmt = Database::connection()->prepare(
            'INSERT INTO customers
            (full_name, phone, email, customer_type, loyalty_points, notes)
            VALUES (:name, :phone, :email, :type, 0, :notes)'
        );

        $stmt->execute([
            'name' => $data['full_name'],
            'phone' => $data['phone'] ?? null,
            'email' => $data['email'] ?? null,
            'type' => $data['customer_type'] ?? 'WALK_IN',
            'notes' => $data['notes'] ?? null,
        ]);

        Response::success([], 'Customer created');
    }
}

```

4M.

backend/app/Controllers/InventoryController.php

```

<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class InventoryController
{
    public function stockSummary(): void
    {
        $stmt = Database::connection()->query(
            'SELECT
                p.id,
                p.product_name,

```

```

        p.product_category,
        si.current_quantity,
        si.reorder_level
    FROM stock_items si
    JOIN products p ON p.id = si.product_id
    ORDER BY p.product_name ASC'
);

Response::success(['stock' => $stmt->fetchAll()]);
}

public function stockIn(): void
{
    $data = Request::body();
    $db = Database::connection();

    $db->beginTransaction();

    $stmt = $db->prepare(
        'INSERT INTO stock_movements
        (product_id, movement_type, quantity, reference_number, notes, user_id)
        VALUES (:product_id, :movement_type, :quantity, :reference, :notes, :user_id)'
    );

    $stmt->execute([
        'product_id' => $data['product_id'],
        'movement_type' => $data['movement_type'] ?? 'PURCHASE',
        'quantity' => $data['quantity'],
        'reference' => $data['reference_number'] ?? null,
        'notes' => $data['notes'] ?? null,
        'user_id' => $data['user_id'] ?? null,
    ]);

    $update = $db->prepare(
        'UPDATE stock_items
        SET current_quantity = current_quantity + :quantity
        WHERE product_id = :product_id'
    );

    $update->execute([
        'quantity' => $data['quantity'],
        'product_id' => $data['product_id'],
    ]);

    $db->commit();

    Response::success([], 'Stock movement recorded');
}
}

```

4N.

backend/app/Controllers/OrdersController.php

```

<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class OrdersController
{
    public function index(): void
    {
        $stmt = Database::connection()->query(
            'SELECT * FROM orders ORDER BY created_at DESC'
        );

        Response::success(['orders' => $stmt->fetchAll()]);
    }

    public function store(): void
    {
        $data = Request::body();
        $db = Database::connection();

        $db->beginTransaction();

        $orderNumber = 'ML-' . time();
    }
}

```

```

$stmt = $db->prepare(
    'INSERT INTO orders
      (order_number, customer_id, waiter_id, table_number, subtotal, discount_amount, total_amount)
    VALUES (:order_number, :customer_id, :waiter_id, :table_number, :subtotal, :discount, :total)'
);

$stmt->execute([
    'order_number' => $orderNumber,
    'customer_id' => $data['customer_id'] ?? null,
    'waiter_id' => $data['waiter_id'],
    'table_number' => $data['table_number'],
    'subtotal' => $data['subtotal'],
    'discount' => $data['discount_amount'] ?? 0,
    'total' => $data['total_amount'],
]);

$orderId = (int) $db->lastInsertId();

foreach ($data['items'] as $item) {
    $itemStmt = $db->prepare(
        'INSERT INTO order_items
          (order_id, product_id, quantity, unit_price, total_price)
        VALUES (:order_id, :product_id, :quantity, :unit_price, :total_price)'
    );

    $itemStmt->execute([
        'order_id' => $orderId,
        'product_id' => $item['product_id'],
        'quantity' => $item['quantity'],
        'unit_price' => $item['unit_price'],
        'total_price' => $item['total_price'],
    ]);
}

$db->commit();

Response::success([
    'order_id' => $orderId,
    'order_number' => $orderNumber,
], 'Order created');
}
}

```

40.

backend/app/Controllers/PaymentsController.php

```

<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;

class PaymentsController
{
    public function cashSettle(int $orderId): void
    {
        $data = Request::body();
        $db = Database::connection();

        $stmt = $db->prepare(
            'INSERT INTO payments
              (order_id, payment_method, amount, transaction_reference, payment_status)
            VALUES (:order_id, :method, :amount, :reference, :status)'
        );

        $stmt->execute([
            'order_id' => $orderId,
            'method' => 'CASH',
            'amount' => $data['amount'],
            'reference' => 'CASH-' . $orderId . '-' . time(),
            'status' => 'SUCCESS',
        ]);

        $update = $db->prepare(
            "UPDATE orders SET order_status = 'PAID' WHERE id = :id"
        );

        $update->execute(['id' => $orderId]);
    }
}

```



```
        Response::success([], 'Cash payment settled');
    }
}
```

4P.

backend/app/Controllers/ReportsController.php

```
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Response;

class ReportsController
{
    public function daily(): void
    {
        $stmt = Database::connection()->query(
            "SELECT
                p.product_category,
                SUM(oi.total_price) AS total_sales
            FROM order_items oi
            JOIN products p ON p.id = oi.product_id
            JOIN orders o ON o.id = oi.order_id
            WHERE DATE(o.created_at) = CURDATE()
            GROUP BY p.product_category"
        );

        Response::success(['daily_sales' => $stmt->fetchAll()]);
    }
}
```

4Q.

backend/app/Controllers/MonitoringController.php

```
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Response;

class MonitoringController
{
    public function overview(): void
    {
        $db = Database::connection();

        $orders = $db->query("SELECT COUNT(*) AS count FROM orders")->fetch();
        $paid = $db->query("SELECT COUNT(*) AS count FROM orders WHERE order_status = 'PAID'")->fetch();
        $stock = $db->query("SELECT COUNT(*) AS count FROM stock_items WHERE current_quantity <=
reorder_level")->fetch();

        Response::success([
            'total_orders' => $orders['count'],
            'paid_orders' => $paid['count'],
            'low_stock_items' => $stock['count'],
        ]);
    }
}
```

Batch 4 Outcome

Backend foundation now includes:

- ✔ boot file
- ✔ database connection
- ✔ response helper
- ✔ request helper
- ✔ router
- ✔ auth controller

- ✓ products controller
- ✓ suppliers controller
- ✓ customers controller
- ✓ inventory controller
- ✓ orders controller
- ✓ payments controller
- ✓ reports controller
- ✓ monitoring controller

Next Smart Move

Build **Batch 5 — PHP Backend Security + Audit Layer**, including:

- auth middleware
- token validation
- role permissions
- audit logger
- approval enforcement
- stock adjustment protection
- cancellation approval protection
- discount approval protection